CRISP-DM Report: Predicting Pull Request Merge Outcomes from GH Archive Events

Nikita Zagainov
n.zagainov@innopolis.university
Data Scientist, ML Engineer

Dmitry Tetkin
d.tetkin@innopolis.university
Business Unit, Project Manager, Business Analyst

May 2026

1 Business Understanding (CRISP-DM Phase 1)

1.1 Determine Business Objectives (1.1)

Background

Open-source software development on GitHub depends heavily on pull requests (PRs) as the main mechanism for proposing, reviewing, and integrating code changes. Maintainers often need to prioritize review attention, while contributors benefit from understanding which early signals are associated with successful integration. Public GitHub event data captures observable workflow signals such as PR opening, PR closure, repository activity, actor activity, comments, pushes, and other event types. It does not capture private repository activity, full maintainer discussions, continuous integration results, or subjective review reasoning. Therefore, the project focuses on observable public signals rather than attempting to fully reproduce human maintainer decisions.

The project studies **pull request merge outcome prediction**. The analytical objective is to estimate whether a PR will eventually be merged using information available at the moment of PR publication and repository or actor activity observed before the opening day.

Business Objectives

- Identify early PR-level, repository-level, and contributor-level signals associated with merge outcomes.
- Build a reproducible model that ranks newly opened PRs by merge likelihood or non-merge risk.
- Provide an interpretable analytical basis for maintainer triage and contributor guidance.

Business Success Criteria

- The model should improve over a trivial majority-class baseline.
- The model should provide useful ranking quality for high-confidence merge or non-merge prioritization.
- The feature set should avoid post-publication leakage and remain explainable to non-technical stakeholders.
- The final conclusions should clearly separate predictive association from causal explanation.

1.2 Assess Situation (1.2)

Inventory of Resources

People.

- Data Scientist / ML Engineer: data extraction, feature construction, modeling, evaluation, and report preparation.
- Business Unit / Project Manager / Business Analyst: business framing, stakeholder interpretation, and presentation/video delivery.

Data. The project uses a local sample of GH Archive hourly event records from July 2015. The raw data consists of compressed JSON-lines archives of public GitHub events. These records were converted into a flat analytical event table for efficient inspection and modeling. The converted event table contains 9,269,533 public GitHub events.

Tools. Python, Polars, pyarrow, scikit-learn, a notebook environment, and matplotlib were used. Polars was used for event processing and feature construction, while scikit-learn was used for model fitting and evaluation.

Requirements, Assumptions, and Constraints

Requirements.

- Use only public event data available from GH Archive.
- Construct a reproducible PR-level modeling table.
- Avoid target leakage from PR closure or future activity.
- Report results using classification and ranking metrics rather than accuracy alone.

Assumptions.

- Public GitHub events are sufficiently informative to capture part of the PR merge process.
- A closed PR with `pull_request.merged=true` represents successful integration.
- Historical repository and actor activity observed before the PR opening day can be used as context available at publication time.

Constraints.

- The data window is limited, so repository and actor histories are left-censored at the beginning of the sample.
- Only PRs whose opening and closing events are both observed in the downloaded window can receive a clean supervised label.
- GH Archive stores event payloads with event-specific JSON schemas, so not all potentially useful GitHub fields are uniformly available.
- The analysis is offline and cannot validate real maintainer decisions through live A/B testing.

Risks and Contingencies

- **Target leakage risk.** Closure-time fields may accidentally enter the feature matrix. Mitigation: exclude closure timestamps, raw merge labels, time-to-close diagnostics, and closure event identifiers from model features.
- **Sample bias risk.** Requiring both opening and closing events favors PRs that complete inside the observation window. Mitigation: interpret the task as short-window observable-lifecycle PR modeling and document this limitation.

- **Class imbalance risk.** Most observed PRs are merged. Mitigation: compare against a majority baseline and use PR-AUC, ROC-AUC, precision@k, and risk-ranking metrics.
- **Historical truncation risk.** Early-window PRs have incomplete prior history. Mitigation: use conservative previous-day history features and state this as a limitation.

Terminology

Table 1: Project terminology.

Term	Meaning
GH Archive	Public archive of GitHub event stream records.
Pull request	GitHub object proposing code changes to a repository.
Merged PR	Closed PR whose payload reports `pull_request.merged=true`.
Non-merged PR	Closed PR whose payload reports `pull_request.merged=false`.
Risk score	$1 - P(\text{merged})$, used to rank PRs by non-merge risk.
Actor history	Aggregated previous public activity of the PR opener.
Repository history	Aggregated previous public activity of the target repository.

Costs and Benefits

The main project cost is analyst and engineering time required to acquire event archives, convert nested event records into analytical tables, construct a PR-level supervised dataset, and evaluate models. No paid data sources or commercial software were required. The potential benefit is an interpretable prioritization signal for maintainers: instead of treating all newly opened PRs equally, a dashboard or report could highlight PRs with unusually high non-merge risk for earlier inspection. In the course setting, the benefit is analytical rather than directly financial.

1.3 Determine Data Mining Goals (1.3)

Data Mining Goals

- Convert public GitHub event streams into a PR-level supervised learning dataset.
- Predict whether a PR will be merged as a binary classification task.
- Compare simple and non-linear models under the same train/test design.
- Identify which pre-opening signals contribute most to merge-outcome ranking.

Data Mining Success Criteria

- Improve ROC-AUC and PR-AUC over the majority-class baseline.
- Improve top-k ranking quality over the baseline merge rate.
- Identify high-risk non-merged PRs with precision meaningfully above the base non-merge rate.
- Keep all features compliant with information available at PR publication time.

1.4 Produce Project Plan (1.4)

Project Plan

Table 2: Project stages, activities, and outputs.

Stage	Key activities	Outputs	Est. time
Data Understanding	Convert raw events, inspect event types and PR payloads	Event and PR summaries	5 days
Data Preparation	Extract PR lifecycles, construct target, build safe features	PR-level dataset	10 days
Modeling	Train baseline and stronger classifiers	Candidate models	7 days
Evaluation	Compare metrics, inspect ranking quality and limitations	Final conclusions	5 days
Deployment	Organize report, code, and recommended use scenario	Course deliverable	3 days

Initial Assessment of Tools and Techniques

Polars was selected for event processing because the raw archive contains millions of JSON-line records and requires efficient columnar aggregation. Scikit-learn was selected for modeling because the task is tabular binary classification and the course objective emphasizes interpretable baselines and reproducible evaluation. Logistic regression was selected as a linear baseline, while random forest and histogram-based gradient boosting were selected as stronger non-linear alternatives.

2 Data Understanding (CRISP-DM Phase 2)

2.1 Collect Initial Data (2.1)

The initial data consisted of hourly GH Archive files downloaded for a continuous July 2015 window. Each raw file is a compressed JSON-lines archive of public GitHub events. Because the raw records are nested and event-type specific, they were converted into a flat analytical event table with common top-level fields and the original event payload preserved as JSON text.

The converted event table contains 9,269,533 events. The common fields retained for inspection were event identifier, event type, timestamp, actor identifier and login, repository identifier and name, optional organization information, and the original event payload.

2.2 Describe Data (2.2)

The dataset is an event stream rather than a conventional relational database. Each row represents one public GitHub event. The main fields used during inspection were:

- event identifier;
- event type, such as `PushEvent`, `CreateEvent`, or `PullRequestEvent`;
- event timestamp;
- GitHub actor identifier and login;
- repository identifier and name;
- event-specific payload.

The sample contains 14 distinct event types. Table 3 summarizes the most frequent event types observed during inspection.

Table 3: Main event types in the converted GH Archive sample.

Event type	Count
`PushEvent`	4,417,737
`CreateEvent`	1,329,287
`IssueCommentEvent`	886,068
`WatchEvent`	826,509
`PullRequestEvent`	470,260
Other event types	1,339,672

The project focuses on `PullRequestEvent`, because it contains both PR opening and PR closure actions. The PR subset contains 470,260 event rows.

2.3 Explore Data (2.3)

The first exploration step checked the action distribution within `PullRequestEvent`. Table 4 shows that the sample contains enough opened and closed PR actions for supervised learning.

Table 4: Pull request event actions.

Action	Count
`opened`	238,541
`closed`	228,072
`reopened`	3,647

The second exploration step checked whether closed PR events contain the merge label. Closed PR events include `pull_request.merged`, allowing the project to distinguish merged

PRs from closed-but-unmerged PRs. Table 5 shows the observed target distribution among closed PR events.

Table 5: Merge label distribution in closed PR events.

Closed PR label	Count
Merged	186,811
Not merged	41,261

Finally, PR lifecycle coverage was checked by grouping PR events by PR identifier. The inspection found 268,072 unique PR identifiers in opened/closed events, including 195,611 PRs with both an opening and a closing event observed in the downloaded data. This confirmed that a supervised lifecycle table could be constructed.

2.4 Verify Data Quality (2.4)

The main data quality issue is not missing tabular values in the usual sense, but event-stream completeness and payload consistency. Several checks were performed:

- **Identifier availability.** PR identifiers, repository identifiers, and PR numbers were available in the relevant PR payloads.
- **Label availability.** The merge label was available for closed PR events.
- **Lifecycle validity.** Only PRs with an observed opening and a later observed closing event were retained for the supervised dataset.
- **Payload variability.** Some modern GitHub fields were absent or uninformative in the 2015 archive sample. In particular, the extracted author association field was constant after extraction and was not used for modeling.
- **Temporal leakage.** Closure-time fields were kept for target construction and diagnostics, but excluded from the model feature matrix.

Table 6: Summary of data quality checks.

Check	Status	Notes
Event volume	Valid	9,269,533 events after conversion.
PR event availability	Valid	470,260 `PullRequestEvent` records.
Lifecycle coverage	Valid with restriction	195,425 usable PR-level records after lifecycle filtering.
Merge label	Valid	Available in closed PR payloads.
Author association	Not useful	Constant in the final table.
Temporal leakage	Controlled	Same-day future activity and closure-time information were excluded from features.

The data was considered suitable for modeling after reducing the event stream to clean PR lifecycle records and constructing features using only opening-time and previous-day information.

3 Data Preparation (CRISP-DM Phase 3)

The purpose of data preparation was to transform the raw GH Archive event stream into a single supervised table suitable for pull request outcome prediction. The chosen analytical grain was one row per pull request. This grain follows directly from the business and data mining goals: the project predicts whether a newly opened PR will eventually be merged.

3.1 Select Data

The starting point was the converted analytical event table containing 9,269,533 public GitHub events. From this table, all `PullRequestEvent` rows were selected for target construction and PR-level metadata extraction. The remaining event types were not discarded entirely; they were used to construct repository and actor history features before PR opening.

For the supervised target, only PRs satisfying the following conditions were retained:

- an `opened` PR event was observed;
- a later `closed` PR event was observed;
- the closed event contained `pull_request.merged`;
- the closing timestamp was later than the opening timestamp.

This selection produced 195,425 PR-level records. The selection excludes PRs that were opened before the sample window, closed after the sample window, or did not have a valid observed lifecycle in the downloaded data. This restriction makes the target clean, but limits the interpretation to PRs whose lifecycle is observable in the sample.

3.2 Clean Data

The raw event archive was converted from compressed JSON-lines files to a flat analytical table. During this conversion, common top-level fields were extracted and the nested payload was preserved as JSON text. This avoided unstable nested schemas across event types while keeping the original payload available for PR-specific extraction.

In the PR lifecycle table, duplicated or repeated events were handled by sorting events by timestamp and taking the first observed opening and first observed closing event per PR identifier. Records where the closing timestamp was not later than the opening timestamp were removed. Closure-time fields were retained only for target construction and diagnostics. They were explicitly excluded from modeling to avoid leakage.

Several fields were checked for usability. The extracted author association field had only one value in the final PR table, so it was excluded from the model. Numeric PR metadata such as commits, additions, deletions, and changed files had no missing values in the final lifecycle table.

3.3 Construct Data

The target variable was constructed as:

$$\texttt{merged} = \begin{cases} 1, & \text{if the closed PR event has } \texttt{pull\_request.merged=true}, \\ 0, & \text{if the closed PR event has } \texttt{pull\_request.merged=false}. \end{cases}$$

The final table had a merge rate of 0.8574, meaning that most observed PRs were merged. This imbalance was later handled by using baseline comparison and ranking-oriented metrics.

The opening-time feature group included:

- commits, additions, deletions, and changed files;
- opening hour, opening weekday, and weekend flag;
- draft flag;
- log-transformed versions of size features and total changed lines.

Repository history features were constructed from all events before the PR opening day. These included cumulative previous-day repository event volume, previous PR event volume, previous issue-comment volume, previous push-event volume, and cumulative repository actor-days.

Actor history features were constructed analogously for the PR opener. These included previous actor event volume, previous PR activity, previous issue-comment activity, previous push-event activity, and cumulative actor repo-days.

The use of previous-day aggregation is conservative: for a PR opened on a given date, all events from that same date are excluded from history features. This prevents events occurring after PR publication from entering the model.

3.4 Integrate Data

The final PR-level table was produced by joining three components:

1. the PR opening table containing PR identity and publication-time metadata;
2. the PR closure table containing only the target and closure diagnostics;
3. previous-day repository and actor history summaries.

The resulting table contains 195,425 PR records, 59,624 unique repositories, and 62,011 unique PR-opening actors. Median observed time to close was 0 hours and the 90th percentile was 45 hours. The zero median reflects that many PRs in the sample are opened and closed quickly, which is plausible for small fixes, automated updates, or repository-internal workflows.

3.5 Format Data

The final modeling dataset was stored as a PR-level analytical table. Modeling used only non-leaky feature columns. The following columns were excluded from model inputs:

- raw PR, event, repository, and actor identifiers;
- target and target-derived fields;
- closure-time diagnostics such as closing timestamp and time-to-close;
- constant or uninformative fields such as author association.

For linear models, numeric features were median-imputed and standardized. For tree-based models, numeric features were median-imputed without scaling. The final modeling matrix contained 195,425 rows and 26 numeric feature columns.

4 Modeling (CRISP-DM Phase 4)

This phase used the prepared PR-level table to train and compare binary classification models for predicting whether a pull request will be merged. The goal was not only to maximize classification metrics, but also to test whether opening-time and pre-opening activity features contain useful signal beyond the majority-class baseline.

4.1 Select Modeling Technique

The task was formulated as binary classification:

$$P(\texttt{merged} = 1 \mid X),$$

where X contains PR opening metadata and repository/contributor activity observed before the opening day. Four model families were selected:

- **DummyClassifier**, using the majority class as a trivial baseline.
- **Logistic Regression**, as an interpretable linear baseline with class weighting.
- **Random Forest**, as a non-linear ensemble model robust to feature interactions.
- **Histogram-based Gradient Boosting**, as a stronger non-linear model for tabular data.

The main modeling assumption is that the selected features are available at PR publication time. Closure-time information and future same-day events were excluded. Since the positive class is dominant, model quality was assessed with threshold-free and ranking metrics in addition to default-threshold classification scores.

4.2 Generate Test Design

The prepared dataset contained 195,425 PR records. A stratified train/test split was used with 75% of records for training and 25% for testing. The resulting split contained 146,568 training records and 48,857 test records. The merge rate remained stable across the split: approximately 0.8574 in both train and test sets.

The selected evaluation metrics were:

- ROC-AUC, to evaluate ranking quality across both classes;
- PR-AUC, to evaluate precision-recall performance under class imbalance;
- F1, precision, and recall at the default decision threshold;
- precision@10%, to evaluate whether the top-ranked merge candidates are actually merged;
- risk precision@k, to evaluate the model's ability to identify PRs at high risk of not being merged.

4.3 Build Model

All models used the same safe feature set. The feature matrix contained 26 numeric predictors: opening-time PR size and timing features, previous-day repository history features, previous-day actor history features, and a small number of derived ratios and log-transformed size variables.

The histogram-based gradient boosting model used 200 boosting iterations, learning rate 0.05, maximum 31 leaf nodes, and L2 regularization 0.1. The random forest used 200 trees, maximum depth 12, minimum leaf size 50, and balanced subsampling. Logistic regression used standardized features and class weighting.

4.4 Assess Model

Table 7 summarizes the held-out model comparison. The histogram-based gradient boosting model achieved the strongest ROC-AUC and PR-AUC, so it was selected as the main model for interpretation and evaluation.

Table 7: Held-out model comparison.

Model	ROC-AUC	PR-AUC	F1	Precision	Recall	Precision@10%
hist_gradient_boosting	0.7324	0.9360	0.9316	0.8745	0.9967	0.9785
random_forest	0.7271	0.9351	0.8434	0.9085	0.7869	0.9781
logistic_regression	0.6547	0.9119	0.7828	0.8950	0.6956	0.9527
dummy_most_frequent	0.5000	0.8574	0.9232	0.8574	1.0000	0.8565

The majority-class baseline obtains a high F1 score because the dataset is dominated by merged PRs. This confirms that accuracy-like metrics are insufficient. The stronger models improve ROC-AUC and PR-AUC over the baseline. The best model, histogram-based gradient boosting, reached ROC-AUC 0.7324 and PR-AUC 0.9360, compared with ROC-AUC 0.5000 and PR-AUC 0.8574 for the dummy model.

Permutation importance was computed on the best model using ROC-AUC as the scoring metric. Table 8 shows the top features. The most important signals are repository and actor activity histories rather than raw PR size alone. This suggests that merge outcomes depend strongly on the surrounding repository workflow and contributor context.

Table 8: Top permutation importance features for the selected model.

Feature	Mean importance	Std.
repo_push_events_before_day	0.0644	0.0021
repo_actor_days_before_day	0.0579	0.0021
actor_push_events_before_day	0.0368	0.0023
repo_issue_comment_share_before_day	0.0330	0.0009
actor_pr_events_before_day	0.0171	0.0030
repo_pr_share_before_day	0.0162	0.0016
commits	0.0149	0.0006
repo_events_before_day	0.0132	0.0013
repo_pr_events_before_day	0.0125	0.0020
repo_issue_comments_before_day	0.0105	0.0016

5 Evaluation (CRISP-DM Phase 5)

The evaluation phase connects the technical model assessment to the business objective of PR triage support. The project does not claim to reproduce maintainers' decisions causally. Instead, it evaluates whether public event features available at PR publication time can produce a useful ranking signal for merge likelihood and non-merge risk.

5.1 Evaluate Results

The selected histogram-based gradient boosting model achieved ROC-AUC 0.7324 and PR-AUC 0.9360 on the held-out test set. This is a meaningful improvement over the majority-class baseline, which has ROC-AUC 0.5000 and PR-AUC 0.8574. The best model also reached precision@10% of 0.9785 for the merge class, showing that the top-ranked merge candidates are merged at a substantially higher rate than the overall base merge rate.

The default-threshold confusion matrix is shown in Table 9. It reveals an important limitation: the model is very strong at identifying merged PRs, but weak at recalling the non-merged class under the default threshold. This is expected because the positive class dominates the dataset.

Table 9: Confusion matrix for the selected model at the default threshold.

	Predicted not merged	Predicted merged
Actual not merged	979	5,989
Actual merged	139	41,750

The detailed classification report shows precision 0.8757 and recall 0.1405 for the non-merged class, and precision 0.8745 and recall 0.9967 for the merged class. Therefore, the model should not be interpreted as a balanced hard classifier at threshold 0.5. Its more appropriate use is ranking and prioritization.

Figure 1 shows the predicted merge-probability distributions for merged and non-merged PRs. The classes are not cleanly separable. However, non-merged PRs have a heavier low-probability tail, which explains the model's useful risk-ranking behavior.

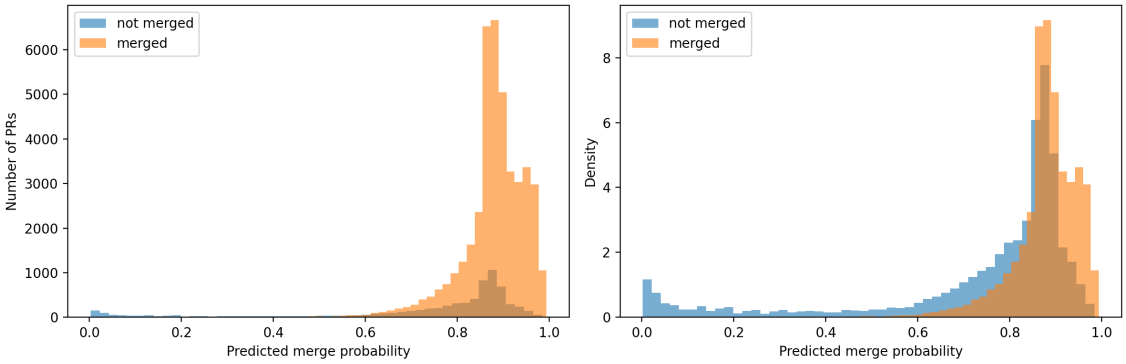

Figure 1: Predicted merge probability distribution for merged and non-merged PRs.

Because non-merged PRs are the rarer and more operationally interesting class, additional evaluation was performed using the risk score

$$\text{risk score} = 1 - P(\text{merged}).$$

Table 10 shows that the model is effective at identifying the highest-risk PRs. The baseline non-merge rate is only 0.1426, but among the top 1% highest-risk PRs, 0.9939 are actually

non-merged. Even in the top 10% highest-risk group, the non-merge precision is 0.4581, more than three times the baseline rate.

Table 10: Risk precision@k for identifying non-merged PRs.

Risk-ranked subset	Non-merge precision	Baseline non-merge rate
Top 1%	0.9939	0.1426
Top 5%	0.6257	0.1426
Top 10%	0.4581	0.1426
Top 20%	0.3325	0.1426

Threshold analysis further supports this interpretation. Table 11 shows that increasing the risk threshold gives higher precision but lower recall. For example, at risk threshold 0.2, the model flags 15.0% of PRs, with non-merge precision 0.3827 and recall 0.4028. At threshold 0.5, it flags 2.3% of PRs, with precision 0.8757 but recall 0.1405.

Table 11: Selected risk-threshold operating points.

Risk threshold	Flagged share	Precision	Recall	F1
0.10	0.6659	0.1876	0.8760	0.3091
0.20	0.1501	0.3827	0.4028	0.3925
0.50	0.0229	0.8757	0.1405	0.2421
0.90	0.0083	1.0000	0.0584	0.1104

The business interpretation is therefore clear: the model is not a universal automatic decision system, but it is useful as a prioritization tool. It can surface a small subset of PRs that are unusually likely to remain unmerged, which could help maintainers inspect blocked, low-fit, or potentially problematic contributions earlier.

5.2 Review Process

The project followed the intended CRISP-DM logic, but the actual work was necessarily data-driven. The main technical uncertainty was whether GH Archive payloads contained enough information to construct PR lifecycles and reliable merge labels. Data understanding confirmed that `PullRequestEvent` contained both opening and closure actions and that closed PR payloads contained the merge label.

The strongest process decision was to define the analytical grain as one row per pull request. This made the target, features, and business interpretation consistent. The second important decision was to enforce a leakage-aware feature policy: all closure-time fields and same-day future events were excluded from the feature matrix. The third important decision was to evaluate the model as a ranking tool, not only as a hard classifier.

Several limitations remain:

- **Window limitation.** The model uses only PRs opened and closed within the downloaded archive window.
- **Left-censoring.** Repository and actor history are incomplete at the beginning of the sample.
- **Public-data limitation.** Private discussions, CI results, review decisions, and maintainers' subjective reasoning are not available.
- **Class imbalance.** Most observed PRs are merged, so default-threshold classification is biased toward the positive class.

- **Causal limitation.** The model identifies predictive associations but does not prove why maintainers merge or reject PRs.

No major omitted step invalidates the current results. The most important limitation is scope: the model should be presented as a proof-of-concept ranking model for PR outcome risk, not as a production-ready maintainer decision system.

5.3 Determine Next Steps

The current model is approved as an analytical result because it improves over the baseline and provides strong risk-ranking signal. It should not be deployed as an automated decision-maker.

Possible continuations are:

1. **Use a longer archive window.** This would reduce lifecycle truncation and improve repository/actor history quality.
2. **Use exact timestamp history.** Instead of previous-day aggregation, features could use rolling windows ending exactly at PR opening time.
3. **Add early interaction features.** A second scenario could predict after the first 6 or 24 hours of activity, using comments and reviews after publication but before final closure.
4. **Apply time-based validation.** Training on earlier days and testing on later days would better approximate deployment.
5. **Connect to richer GitHub data.** Review comments, CI statuses, labels, file types, and textual PR descriptions could improve explanatory power.

The selected decision is to proceed with the current report as the final course deliverable. The model is useful for ranking PR merge outcomes under a controlled leakage-aware setup, while the limitations are explicit enough to avoid overstating the result.

6 Deployment (CRISP-DM Phase 6)

The project does not include production deployment. In this course setting, deployment means organizing the analytical result, code, figures, and conclusions in a form that can be reviewed and reused. This is consistent with a report-based deployment scenario: the model is not embedded into a live GitHub workflow, but the result is presented as a reproducible decision-support analysis.

6.1 Plan Deployment

The recommended deployment is analytical and report-based rather than automatic operational deployment. The model should be used as a decision-support tool for ranking newly opened pull requests by merge likelihood or non-merge risk. A possible operational scenario is a maintainer dashboard that displays:

- a predicted merge probability;
- a non-merge risk score;
- the most important feature groups, such as repository activity and contributor history;
- warnings about data limitations and model scope.

The model must not be used to automatically reject, hide, or deprioritize contributors. Its output should support human triage, not replace maintainer judgment. The safest practical use is to surface potentially blocked or low-fit PRs for earlier inspection.

6.2 Plan Monitoring and Maintenance

If the approach were used beyond the course setting, monitoring would be required. The following issues should be tracked:

- **Data drift:** GitHub workflows, repository practices, and contributor behavior can change over time.
- **Schema drift:** event payload fields may change or become unavailable.
- **Target drift:** the base merge rate can vary by time period, repository type, and project maturity.
- **Performance drift:** ROC-AUC, PR-AUC, and risk precision@k should be recalculated on newer time windows.
- **Ethical use:** the score should be monitored to ensure that it does not become an automatic gatekeeping mechanism against contributors.

A production version should retrain the model on newer and longer archive windows, evaluate with time-based validation, and validate feature extraction after any schema changes. If risk precision drops close to the baseline non-merge rate, the model should be disabled or retrained.

6.3 Produce Final Report

The final deliverable of this project is the written CRISP-DM report together with the accompanying code notebooks and generated artifacts. The report summarizes the business objective, data understanding, data preparation pipeline, model comparison, evaluation results, limitations, and recommended next steps.

The final presentation and video are handled separately by the project team. They should present the same central conclusion as the report: public event data available at PR publication time contains useful ranking signal, but the model should be treated as a triage-support tool rather than an automatic decision system.

6.4 Review Project

The project successfully produced a leakage-aware PR-level dataset and a ranking model that improves over the majority-class baseline. The main lessons are:

- GH Archive is suitable for event-level workflow mining, but the observation window strongly affects lifecycle completeness.
- PR merge prediction is heavily imbalanced because most observed PRs are merged.
- Default-threshold classification is less informative than ranking-based evaluation for this task.
- Repository and contributor history features provide stronger signal than raw PR size alone.
- The result is useful for prioritization, but not sufficient for automated maintainer decisions.

The main improvement for future versions is to use a longer time range and exact timestamp-based historical windows. This would reduce left-censoring and approximate a real deployment setting more closely.